Telemetry Blindness at the Agent–Business Boundary: A Controlled Measurement of Output-Contract Enforcement

Estefano Senhor Ferreira
Senior Software Engineer
estefano.senhor@gmail.com · `https://github.com/estefano-ferreira/authored-plan-agents`

Draft — extracted from repository state `v0.4.0`, 2026-08-06

Abstract

LLM agents that write business state can report success while persisting corrupted records. We reproduce this failure as a controlled six-cell comparison (plus a control arm) — *decoding constraint* (provider-enforced response schema vs. schema stripped) crossed with *boundary policy* (tolerant repair vs. a strict retry-once-then-fail output-contract guard), a fault injector forcing format violations in the unconstrained cells and, in two pre-registered follow-up cells, downstream of the active schema — on one codebase, with the main comparison on one model. Pre-registered follow-up arms replicate cells A and B across two further model configurations and a second system-of-record engine (reported in the limitations). Ground truth is read directly from the persisted database of an external system of record. Two findings carry the paper. First, *telemetry blindness*: with repair positioned before the contract check, the tolerant configuration reported success 10/10 and persisted ten garbage business records while every platform-side instrument — status, contract counters, repair count — read success or no violation; only the database showed the corruption. The signature reproduced unchanged with the schema active and the fault injected downstream of it — a pre-registered follow-up whose opposite prediction (that fence-stripping repair would recover the valid content under the fence) the data falsified. Second, the guard's price sheet: under enforced decoding both boundary policies were byte-equivalent and the guard's idle cost was zero (natural violations 0/10 — a bounded observation, not an elimination claim); when a violation does arrive, the strict guard's price is one extra model call, buying typed, unpersisted failure instead of silent corruption — a trade a naturally occurring provider schema-dialect mismatch exercised for us. The implication is deliberately narrow: constrained decoding is the suppression mechanism; the architectural boundary is what keeps the channel closed when that simpler mechanism silently breaks. All evidence is version-controlled and inspectable.

1 Introduction

The failure this paper measures was first observed in the wild, not designed. Ten consecutive executions of an email-to-ERP business flow reported success; ten times, the ERP received a truncated, markdown-fenced JSON blob as the customer-facing `summary` field – indistinguishable from a legitimate record to any downstream consumer. The platform's contract counters read zero violations: a tolerant parser "repaired" each response before any check could see it. Every authorization held. The business API's own validation passed — the corruption was in *content*, and content of valid form.

The gap has a name older than agents. Transaction processing drew this line four decades ago: in the paper that coined ACID, a committed transaction "by definition commits only legal results" — and, in the same discussion, "since, by definition, each transaction is correct, the effects of an inevitable incorrect transaction (i.e., the transaction containing faulty data) can only be removed by countertransactions" [1]. Consistency guarantees legality by the database's lights; correctness of the written content is *assumed*, not guaranteed. What has changed is the writer, not the guarantee. Under programmatic writes the gap is rarely exercised: code writes what the programmer specified, so a semantically wrong value of valid form is the exception. Under LLM generation, form is enforced at the decoder while content is probabilistic — the gap goes from rarely exercised to routinely exercised. That is why business validity, which could remain an implicit programmer responsibility, needs an explicit owner when the content author is stochastic.

The phenomenon is not idiosyncratic. Advani measures *false success* — an agent claiming completion with no corresponding state change in the system of record — at 45–48% of failures in single-control τ^2-bench domains, and finds LLM judges fail to detect it (AUROC ≤ 0.65) [2]. Cao et al. find 27–78% of benchmark-reported successes to be procedurally corrupt [3]. SABER localizes the damage: deviations in *mutating* steps reduce task-success odds by up to 92–96%, while deviations in non-mutating steps have little effect [4] — the write path is where failures become decisive.

What the literature does not settle is an engineering question. Provider-enforced structured output (constrained decoding) now exists and demonstrably works — so the question that can actually be answered with measurements is conditional: *when decoding enforcement silently fails, what does each boundary policy between generated content and the business write do with the violation — and what does keeping the stricter policy cost while decoding still works?* This paper answers with a controlled measurement whose ground truth is not the agent's report, not the platform's telemetry, but rows in the business database.

Contributions. (1) A controlled comparison of six boundary configurations — decoding enforcement crossed with boundary repair policy, a fault injector in the unconstrained column and (pre-registered) downstream of the active schema, and a shared control arm — with ground truth read from a persisted external system of record. The design is not a complete factorial (§3). State-grounded scoring itself is established practice — deterministic evaluation against the persisted environment is how STATE-Bench scores state-mutating tasks [8] and what GroundEval provides as an LLM-judge replacement [9]; what this comparison adds is the *question* asked of that ground truth: not whether the agent reached the correct final state, but what each boundary policy did to the *content* of the records it persisted — a mechanism ablation reading record quality row by row, priced on both sides. (2) The measured finding that *tolerant repair conceals rather than repairs*, and specifically that it blinds the platform's own integrity counters — the audit trail records zero violations in the cell that persists 100% garbage: the *telemetry blindness* defined in §4. (3) An honest pricing of the boundary guard as insurance: idle cost zero when decoding enforcement works, one extra model call per violation when it does not, and a naturally occurring configuration failure in which the insurance paid out.

2 The measured system

The reference implementation is a minimal agent platform for business process automation (three agents, eight tools, two human-authored plans) in which non-determinism is confined to three model decisions: plan selection, capability selection, and content generation. Step order is declared in versioned YAML; the model never decides what runs next. Three properties matter for this experiment:

1. **External system of record.** Business writes go through an ERP service (SQLAlchemy over SQLite) that owns its invariants as database constraints and rejects violations with typed errors. The platform holds no business entity. Ground truth for every cell is read from this database, never from platform state.

2. **Output contract on the write path.** The *boundary* is the single interface through which generated content becomes business state. Generated content that feeds a business

write is validated against a declared schema at a boundary the generator cannot bypass; on violation, at most one retry, then a typed failure — nothing unvalidated persists. This is the mechanism under test.

3. **Unsampled audit.** Proposals, policy decisions, business validations, output-contract events and compensations are recorded append-only, as distinct kinds. The experiment measures, among other things, whether this trail can be trusted — it cannot, in one cell, for a reason worth stating precisely (§4).

The pattern's full specification and its seven load-, registration- and runtime-enforced restrictions are in the repository; none of them is the subject here. The subject is the boundary between generated content and persisted business truth.

3 Experimental design

Axes. *Decoding constraint*: the provider-enforced JSON schema (`responseSchema`) either passed through to the model provider, or stripped by an infrastructure decorator (`SchemaStrippingClient`). *Boundary policy*: either a strict output-contract guard (`retry_once_then_fail`) or a tolerant repair decorator (`TolerantRepairClient`) that strips markdown fences well enough to satisfy the parser and passes the result through — an emulation, as an infrastructure decorator, of the historical fallback behavior that produced the original incident. Construct validity requires one precision: the historical fallback lived *inside the capability's parser* (after a parse failure it stuffed the fenced blob into the business field); the emulation reconstructs that behavior as an infrastructure decorator so that measurement never touches the platform's behavior layer. The comparison is therefore against a reconstruction at a different layer — and what transfers is positional, not layer-specific: any absorption upstream of the contract check blinds the counters the same way (Figure 2).

Fault injection. Cells A and B (no schema) additionally run a generation-fault injector (`GenerationFaultInjectionClient`) that re-fences *every* generation response. The violation rate in these cells is therefore forced by construction — 10/10 measures the injector, not the model. This is deliberate: the natural no-schema violation rate on this model was measured in an earlier round (10/10 fenced — itself an artifact of flash-tier defaults under prompt-only control), and the question this matrix asks is conditional: *given* a violation, what does each configuration do with it? The injector also re-fences the retry attempt, so recovery in cell B is impossible by construction; the guard's failure path, not its recovery path, is what B measures.

Stated plainly: because the injector runs only in the no-schema cells, **the design is not a complete factorial** — the fault arm co-varies with the decoding axis. Cells C/D measure the schema configuration under *natural* conditions only; within the original four cells, no configuration subjected active decoding enforcement to a forced violation — the pre-registered follow-up cells E/F later did exactly that (fault injected *downstream* of the active schema; results in §6). The findings are worded accordingly (F3 claims invisibility of the repair axis and zero idle cost, not that decoding "holds under attack").

Cells and control. Ten repetitions per cell of the two-agent email-to-ERP plan (the only plan with a generation step), each against a fresh SQLite file. A control arm runs the D configuration on the no-generation scheduling plan (3 repetitions): the axes should touch nothing but the generation step. Model: `gemini-3.1-flash-lite`, same prompts throughout; everything varies by command-line flags only.

Cell	Configuration	Status (10 reps)	Viol./Retry/Fail	Repairs	Rows	Valid	Cost
A	no schema + fault + tolerant	`completed` 10/10	0/0/0	10	10	**0**	$0.00304
B	no schema + fault + strict	`failed_clean` 10/10	10/10/10	0	**0**	—	$0.00509
C	schema + tolerant	`completed` 10/10	0/0/0	0	10	10	$0.00247
D	schema + strict	`completed` 10/10	0/0/0	0	10	10	$0.00246
ctrl	D config, no-generation plan	`completed` 3/3	0/0/0	0	3	n/a	$0.00017
E	schema + fault + tolerant	`completed` 10/10	0/0/0	10	10	**0**	$0.00245
F	schema + fault + strict	`failed_clean` 10/10	10/10/10	0	**0**	—	$0.00384

Table 1: The integrity matrix. "Rows" are records created in the external ERP database; "Valid" counts rows whose persisted `summary` is clean one-line text (verified by direct SQL inspection). Cell A's ten rows each carry a double-fenced JSON blob in the business field while every counter the platform exposes reads zero. Below the rule: the pre-registered follow-up cells E and F (§6), run after the original comparison — E reproduces A's signature with the decoding constraint active upstream. Latency means are omitted: they are contaminated by free-tier 429-backoff stalls (worst case 111 s) and carry no signal about the axes.

What is read as the result. Reported status; contract violation/retry/failure counters from the audit trail; repair count; rows created in the ERP database; per-row validity of the persisted `summary` (checked by direct SQL inspection, not via any platform report); tokens and cost.

Price basis. Cost figures are computed by the runner from the per-model pricing table in the platform's observability writer: for `gemini-3.1-flash-lite`, $0.25 per million input tokens and $1.50 per million output tokens, rates as recorded on 2026-07-28 from the official pricing page (`https://ai.google.dev/gemini-api/docs/pricing`). Token counts come from the provider's own `usage_metadata` (`prompt_token_count`, `candidates_token_count` plus thought tokens where the model reports them, `cached_content_token_count`), not from an estimator. Cached input is billed at 10% of the input rate in the runner's formula; in these runs cached tokens were zero — verified by recomputing all 123 recorded per-execution costs (the six-cell comparison, the control arm, and the follow-up arms at their own list rates: `gpt-4o-mini` and Groq-served `gpt-oss-120b`, both $0.15 / $0.60 per million as recorded; the Postgres arm on the baseline model's rates) from their token counts alone, which reproduces every recorded figure exactly (`scripts/verify_costs.py` in the repository). All runs executed inside free-tier quota, so the figures are list-price equivalents, not amounts billed. List prices change; the figures are reproducible against the rates as of the stated date, and the token counts — which do not change — are in the tracked evidence files (`results/integrity-matrix/integrity_matrix.jsonl`, per (cell, rep)).

4 Results

F1 — without the schema, the boundary alone separates the outcomes. Same fault, same model, same pipeline: A reports `completed` 10/10 and persists 10 garbage business records; B reports `failed_clean` 10/10 and persists zero. One epistemic asymmetry must be stated: B's outcome is deterministic by construction (the injector re-fences the retry and the guard is coded to fail typed), so B *verifies*, under real-model conditions, that the strict policy does what its code says — a designed contrast, not a discovery. The discovery is in A, and it is emergent rather than programmed: the corrupted content does not merely evade the check — it reaches the business database while every platform-side instrument reads clean (F2).

	Cell A (no schema, tolerant)	Cell B (no schema, strict)	Cell E (schema, tolerant)
what the platform reports			
Agent report	`completed` 10/10	`failed_clean` 10/10	`completed` 10/10
Contract counters (viol. / retry / fail)	0 / 0 / 0	10 / 10 / 10	0 / 0 / 0
Repair count	10	0	10
what the database holds			
Rows in ERP	10	0	10
Valid rows	**0**	—	**0**

Figure 1: What each observer sees (data from Table 1; no new measurement). The separation between the two bands is the finding: in cells A and E, three platform-side instruments agree the runs succeeded — status `completed`, contract counters at zero, ten "successful" repairs — while the business database holds ten corrupted records and zero valid ones. Cells A and E differ in decoding condition (schema stripped vs. active, the fault injected downstream of it): the same blindness signature in both columns is Proposition 1's positional claim (§4), visible at a glance.

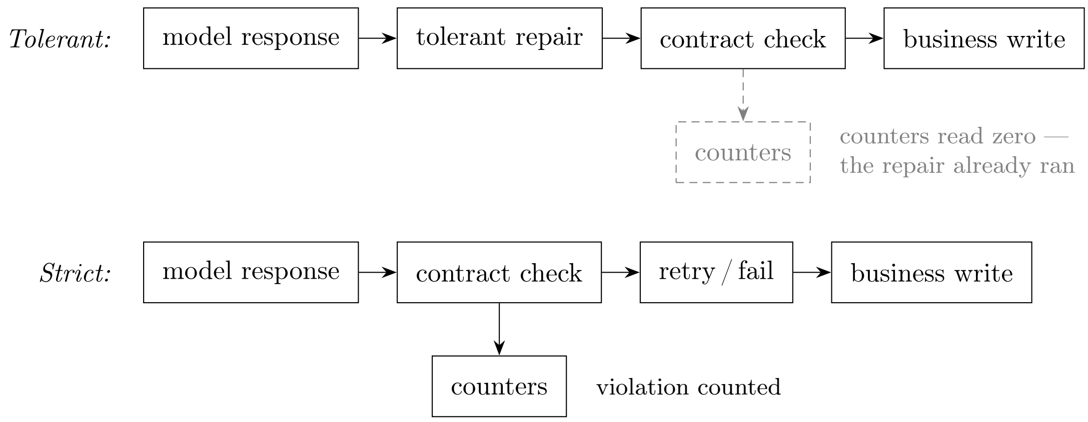

Figure 2: Why F2 happens: one position in the sequence, not a property of repair. In the tolerant pipeline the repair runs before the contract check, so the check — and the counters it feeds — never see the violation; in the strict pipeline the check runs first and the violation is counted before retry or typed failure.

F2 — tolerant repair conceals, and blinds the telemetry. Cell A's contract counters read 0/0/0 because the repair runs *before* the contract check ever sees the response; only the repair count (10) and direct database inspection reveal what happened. The practitioner literature warns in prose that syntactic repair "fixes syntax, not semantics"; part of it celebrates high repair-success rates as a virtue. The matrix quantifies the inverted reading: "repaired" 10/10, valid 0/10. The consequence extends a lesson the false-success literature draws about agent self-reports [2, 3] one level deeper: in a repair-before-check pipeline, *the platform's own integrity telemetry is not evidence either*. Only the system of record is (Figure 1); and the mechanism is positional, not a property of repair itself (Figure 2). Two levels of this claim must be kept apart. The *positional* mechanism is general: any absorption upstream of a check blinds the counters that check feeds. The *measured* result is narrower: format repair ahead of a format check. A repair that normalizes content ahead of a form check, or form repair ahead of a semantic check, was not tested and may behave differently (§6).

Definition 1 (Telemetry blindness). A pipeline exhibits *telemetry blindness* when every platform-side observer reports success or no violation while the system of record holds corrupted state.

Proposition 1 (Sufficient condition). Given a violation that reaches persistence, any absorption step positioned upstream of the check that feeds those observers suffices: the check never sees the violation, so the counters it feeds cannot record it — and the absorption step's own counter reads as successes, not violations. The condition is mechanism-agnostic: it holds for any absorption step, not only the format repair measured here (the measured narrowing above stands).

Observed instance. Cell A: agent report `completed`, contract counters at zero, repair count reporting ten successes, ten corrupted rows in the business database. A second instance was measured after this definition was written and pre-registered (§6): the same signature — counters at zero, ten repairs, ten corrupted rows — with decoding enforcement active upstream and the fault injected downstream of it. The condition is positional, indifferent to what sits upstream of the absorption.

F3 — with the schema, no violation occurred and the repair axis is invisible. Cells C and D are byte-equivalent in outcome (10 clean rows each, ∼same tokens, ∼same cost) with zero natural violations at N=10 — an observation bounded at $\leq 25.9\%$ (§6), not an elimination claim, and these cells never faced a forced violation (§3). The direct evidence that decoding-level enforcement suppresses *natural* violations is the earlier before/after round on the same pipeline: 10/10 violations under prompt-only control → 0/10 with the schema, at −18% cost per completed task. What C/D add is the guard's price sheet: under the enforced-decoding configuration the guard has literally nothing to do, and its idle cost — extra model calls, extra tokens — is zero.

F4 — the guard's price is one extra call per violation. B spent ∼2× A's output tokens ($0.00509 vs. $0.00304 per 10 reps): the retry doubles the generation call and both attempts fault by construction. That is the entire price of knowing.

F5 — the control arm isolates the generation step. The no-generation plan completed 3/3 with clean persisted state under the D configuration: the axes touch nothing else in the pipeline.

5 The naturally occurring case

The matrix's guarded no-schema cell models a configuration class that occurred naturally during the study, unplanned. The first corrected measurement round passed the output schema through the provider's `response_schema` config field — which accepts an OpenAPI-subset dialect and rejected the standard JSON-Schema key `additionalProperties` with a live HTTP 400. Constrained decoding was thereby silently disabled by configuration: exactly the class of provider-side surprise that had previously produced 10/10 silent corruptions. With the boundary guard in place, it produced instead 10/10 *visible, typed* failures with the ERP at zero rows. The fix (the provider's `response_json_schema` field, which accepts real JSON Schema) is one line — and the secondary finding is portable: JSON Schema is not portable verbatim across providers (two Gemini config fields with different grammars; Anthropic requiring `additionalProperties: false` plus `required`; OpenAI's `strict` wrapper), and dialect translation is adapter responsibility.

This is the paper's claim in one incident, stated narrowly: the architectural boundary is *not* the primary suppression mechanism — decoding-level enforcement is what suppressed every natural violation we measured (F3 and the before/after round). The boundary is what keeps the channel closed when the simpler mechanism silently breaks. As insurance it is unusually well priced: idle cost zero (F3), one extra model call per violation when it fires (F4). We observed one naturally occurring claim event; its base rate in production is unknown (n=1), and we state it as an existence proof, not a frequency.

6 Limitations and statistical bounds

Small cells, stated as such. Ten repetitions per cell. A 0/10 observation bounds the underlying rate only to $\leq 25.9\%$ at one-sided 95% confidence ($1 - 0.05^{1/10}$); "zero violations at N=10" is compatible with a material residual rate and is reported as an observation, not an elimination claim. The qualitative cell separations (10 garbage rows vs. zero; counters blind vs. counting) do not depend on rate estimates.

The injector measures the injector. The 10/10 violation rate in A/B is forced; the natural rate on this model under prompt-only control (10/10 in the original round) is an artifact of flash-tier defaults. All conditional claims are of the form "given a violation".

Not a complete factorial — and what the completed arm measured. At the time of the four-cell comparison the fault arm existed only in the no-schema column (§3). The two missing cells — schema + injected fault, under each boundary policy — were subsequently pre-registered (branches and the uncomfortable outcome fixed before any number existed) and run on the same model, 10 repetitions each. **The pre-registered prediction was falsified.** Because the injector corrupts downstream of the provider, the injected fence wraps schema-valid JSON, and we predicted — in the pre-registration, in review, and in an earlier version of this paragraph — that fence-stripping repair would plausibly recover *valid* content there. Measured: it does not extract; it absorbs. The tolerant cell reported `completed` 10/10 and persisted ten fenced blobs into the business field with contract counters at zero and ten repairs reported — telemetry blindness reproduced under an *active* decoding constraint, a second instance of Proposition 1 in a different decoding condition. The strict cell reported `failed_clean` 10/10 with zero rows, refusing content that was, under the fence, valid — and remains the verification it was pre-declared to be (the injector re-fences the retry, so its outcome is deterministic by construction). The counterfactual "tolerance would have recovered it" is therefore measured false for this repair implementation, not conceded hypothetically: the alternative to refusal was not recovery; it was corruption. The design is *still* not a complete factorial: the no-schema/no-fault arm (natural violations under each boundary policy) has never run as a cell — the incident round covers only its tolerant half.

One violation class. The injected fault (markdown re-fencing) is the class the original incident exhibited. Tolerant repair in the wild absorbs other classes (truncation, wrong fields, schema-valid-but-wrong content); semantically wrong content of valid form passes every cell clean — content quality inside a valid schema is explicitly out of scope, and the matrix measures *form* integrity only.

The baseline's raw file is lost. The before/after round cited in F3 rests on a baseline whose raw per-execution file was deleted by a cleanup step before the repository was under version control. Its aggregates survive in the repository's study log, and the corrected round's raw file is tracked; the 10/10 figure is therefore reproducible from surviving aggregates, not from raw records. The repository discloses this loss, and so does this paper.

Model scope — one model in the main comparison, arms beyond it. All cells of the main comparison ran on `gemini-3.1-flash-lite`. A pre-registered cross-model replication of cells A/B has since run its family arm (`gpt-4o-mini`, 10 repetitions per cell, model-suffixed tracked evidence): every repetition that reached the generation boundary reproduced the original reading — tolerant: 6/6 `completed` with fenced garbage persisted and counters at zero (telemetry blindness in a third model family); strict: 1/1 typed failure — and zero rows persisted in all 10 strict repetitions regardless of failure locus. The caveats are part of the result: the strict cell's boundary sample is one, because 13/20 repetitions diverged *upstream* on selection-response format (the model answers a sentence instead of the bare id) — a per-family selection result reported separately per the pre-registration, and itself a falsification of this study's earlier reading that the id-only selection contract is "naturally robust" as a property of the task. A further pre-registered arm (Groq-served `gpt-oss-120b`, open-weights) then provided what the mini arm could not: *every* repetition reached the boundary in both cells (the id-only protocol held in this configuration), giving full boundary samples — tolerant: 10/10 garbage persisted with counters at zero; strict: 10/10 typed failures, zero rows. The mini arm's sample-of-one caveat is thereby resolved by measurement in a different configuration, not withdrawn. The frontier-tier arm (`claude-sonnet-5`, amendment recorded) has not run.

Self-built system of record. The ERP is a mock written by the same author, with two invariants designed to be checkable. The pattern's guarantee delegates wholesale to the system of record's competence; this study cannot measure an incompetent one. A pre-registered engine arm reran cells A/B with the same ERP on PostgreSQL (baseline model held fixed) and reproduced both readings exactly — the results are engine-portable and a SQLite artifact is ruled out — but the circularity itself is untouched: the engine varied, the owner did not. Relatedly, the platform's registration-time checks validate *coherence of declared* tool metadata, not the truth of the declarations.

7 Related work

False and corrupt success. Advani [2] names and measures false success and shows detection failing; Cao et al. [3] measure procedural corruption behind reported success; SABER [4] shows mutating steps are where deviations become decisive. This experiment is a persistence-grounded replication of the phenomenon with a mechanism ablation none of the three performs.

Structured output. Provider-enforced structured output and constrained-decoding libraries are commodity; the known enforcement-level gradient (prompt-only weakest, native structured output strongest) is replicated, not discovered, by our before/after. What we add is the controlled comparison against the boundary policy and the database-grounded reading. The closest experimental neighbor is harness engineering for auditable enterprise agents [5]: contract preservation measured at a replaceable composition boundary, with a fault-injection control confirming the validators flag deliberately broken contracts (270 boundary runs across three hosted models) and an ablation in which code-owned enforcement preserves both safety and full utility where a bolt-on guardrail over-refuses (88/120 vs. 120/120). Their contracts govern the composed *answer* (source grounding, entity routing, output hygiene); ours governs a business *write*, with ground truth read from the persisted store and a repair-policy axis their design does not vary — the telemetry-blindness result (F2) has no counterpart there.

Transaction processing. The distinction this paper leans on is the transaction literature's own: committed results are "legal" by the database's lights, and the transaction containing faulty data sits outside the guarantee, removable only by countertransactions [1]. Cell A is that distinction exercised one layer up — constraints validated, transaction committed, content wrong. The analogy is claimed for the gap only: nothing in this paper provides atomicity, isolation, or durability.

State-grounded evaluation. Judging agents against persisted state rather than their own reports is established: STATE-Bench's deterministic scorer "compares the final environment state to ground truth" for state-mutating tasks over a pre-populated database of business artifacts [8], and GroundEval is, by its own title, a deterministic replacement for LLM-as-judge in stateful agent evaluation [9]. Both ask whether the agent reached the correct state — task scoring. This paper asks a different question of the same ground truth: what each boundary policy did to the content of the records it persisted. The source of truth is shared inheritance; the mechanism ablation over record quality is what this measurement adds.

Gray failure and differential observability. The systems literature named the observability structure underneath our finding in 2017: a system experiences *gray failure* "when at least one app makes the observation that system is unhealthy, but observer observes that system is healthy" — *differential observability*, in a model where the observer is part of the system and the app is external to it [12]. Telemetry blindness is not that situation; it is its degenerate case. In cell A *no* entity observes unhealthy: the platform's observer instruments (status, contract counters, repair count) read clean, and a downstream consumer of the ERP — the model's app — reads records of valid form. The observability table of Huang et al. would place cell A in its no-failure quadrant; the disagreement is not between two observers but between every observer and the persisted state itself, and only direct inspection of the system of record reveals it. We therefore claim telemetry blindness as a specialization of differential observability — the case where the differential has collapsed — not as a new phenomenon. The structure is now actively benchmarked at the trace layer: AgentTelemetry maps fourteen fault types onto agent-specific span kinds and shows a complete span taxonomy lifting fault detection from 0.429 to 1.000 [10]. Telemetry blindness is the case span completeness does not reach: the absorbing step emits a *successful* span by its own contract, so no added span kind turns red — only the content of the persisted record does. At the agent layer, the nearest neighbour is a longitudinal taxonomy of silent failures in a production LLM agent runtime, itself framed in gray-failure terms and citing Huang et al., whose error-swallowing-and-dilution class the absorption step measured here instantiates [13]. Its "fail-plausible" class is a *different* specialization of the same lineage, and the distinction matters: there the failure fabricates narrative that deceives the observer; in telemetry blindness nothing is fabricated — the violation is absorbed upstream and every observer honestly reads success. Otherwise, the difference is method — taxonomy drawn from a production runtime there, controlled measurement of one mechanism here.

Error hiding. The absorption step is family-adjacent to the classic error-hiding (error-swallowing) anti-pattern — catching a failure and continuing without recording it [11] — and the practitioner literature already states that anti-pattern's observational consequence on the availability axis: an API that answers 200 for failures keeps availability dashboards green. The distinction Proposition 1 rests on is one level deeper: in error hiding an error occurs and is swallowed; in the measured absorption *no error occurs at the absorbing step* — the repair succeeds by its own contract and its counter records successes. Telemetry blindness requires no swallowed exception anywhere in the pipeline; this is the same no-fabrication move drawn against fail-plausible above.

Durable execution. Temporal-style runtimes and the agent checkpointers built on them guarantee that an *execution* completes, replays, and survives crashes; they say nothing about whether the persisted *content* is business-true. A durable workflow persists garbage perfectly durably and perfectly auditably — cell A is exactly that artifact. Validity authority is orthogonal to durable execution, not redundant with it.

Adjacent enforcement loci. Intent-governed authorization narrows tool manifests server-side and checks payload effects before execution [6] — the guarantee-belongs-to-the-validator argument on the authorization axis, where ours is on the validity axis. Proof-of-execution frameworks formalize "null effect on deny" and tamper-evident execution records under cryptographic assumptions [7]; our audit trail makes no such claim (F2 is precisely a measurement of how far platform-side records can be trusted — and where they cannot).

Delimitation. Statements of absence in this paper's related-work mapping ("the telemetry-blindness result (F2) has no counterpart there") are search- and time-bound as of 2026-08-06, over a literature that does not index by these axes; during this study's own review rounds, an adjacent paper that predated one of our searches was found only by a later, independent pass. Absence findings mean "not found by a stated search at a stated date", never "does not exist".

8 Conclusion

Between an LLM's output and a business database there is a boundary, and this paper measured what each boundary policy does when a contract violation reaches it — forced by an injector in the unconstrained cells, naturally occurring in the incident and the dialect accident that bracket them. In every round we measured, decoding-level enforcement suppressed all natural violations — and a strict architectural guard behind it costs nothing while that remains true. What the guard buys is the difference between the two remaining worlds: when the decoding constraint silently breaks — a dialect mismatch, a misconfiguration, a provider change — a tolerant pipeline persists garbage while reporting success and zeroing its own counters — the telemetry blindness of §4 — while a strict boundary fails loudly, typed, with nothing persisted. In the combination we measured — format repair ahead of a format check — repair-before-check did not repair; it concealed. What generalizes is the position, not the repair: any absorption upstream of a check blinds the counters that check feeds (Figure 2), and that positional lesson is what travels beyond this stack. The evidence for every sentence above is a version-controlled artifact: per-cell database snapshots, append-only audit trails, and raw model-call logs, inspectable from a fresh clone.

Data availability. Repository tag `v0.4.0`: implementation, restriction-enforcing test suite (18 passed, 7 strict-xfail anti-vacuity probes), and tracked evidence under `results/integrity-matrix/` (checkpointed JSONL, consolidated report, per-cell SQLite snapshots `matrix-{A,B,C,D,control,E,F}.sqlite` plus the model- and engine-suffixed arm snapshots (`-gpt-4o-mini`, `-openai-gpt-oss-120b`, `-pg`), audit trails) and `results/runs/` (the naturally occurring dialect-failure round, `run-20260727-215429.json`; the corrected round, `run-20260727-215811.json`). One caveat carried from §6: the baseline round's raw file (`run-20260727-212338.json`) does not exist — lost to a pre-versioning cleanup; its aggregates survive in the repository's study log.

References

[1] T. Härder and A. Reuter. Principles of transaction-oriented database recovery. *ACM Computing Surveys*, 15(4):287–317, 1983.

[2] R. Advani. From confident closing to silent failure: Characterizing false success in LLM agents. arXiv:2606.09863, 2026.

[3] Y. Cao et al. Beyond task completion: Revealing corrupt success in LLM agents through procedure-aware evaluation. arXiv:2603.03116, 2026.

[4] A. Cuadron, P. Yu, Y. Liu, and A. Gupta. SABER: Small actions, big errors — safeguarding mutating steps in LLM agents. arXiv:2512.07850, 2025. Under review, ICLR 2026.

[5] J. Ahn and M. Kim. From prompts to contracts: Harness engineering for auditable enterprise LLM agents. arXiv:2607.08028, 2026.

[6] G. Zhu and C. Wang. Intent-governed tool authorization for AI agents. arXiv:2606.22916, 2026.

[7] J. Rhodes and G. Kang. Proof of execution: Runtime verification for governed AI agent actions. arXiv:2607.05397, 2026.

[8] Microsoft. Introducing STATE-Bench: A benchmark for AI agent memory. Microsoft Open Source Blog, 2026-05-19. Accessed 2026-08-06.

[9] GroundEval: A deterministic replacement for LLM-as-judge in stateful agent evaluation. arXiv:2606.22737, 2026.

[10] AgentTelemetry: A fault detection benchmark and toolkit for LLM agent observability. In *Proceedings of the 3rd ACM International Conference on AI-Powered Software*, 2026.

[11] Error hiding. Wikipedia. Accessed 2026-08-06.

[12] P. Huang, C. Guo, L. Zhou, J. R. Lorch, Y. Dang, M. Chintalapati, and R. Yao. Gray failure: The Achilles' heel of cloud-scale systems. In *Proceedings of HotOS*, 2017.

[13] W. Wu. When errors become narratives: A longitudinal taxonomy of silent failures in a production LLM agent runtime. arXiv:2606.14589, 2026.